Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №7
По дисциплине «Современный платформы программирования»
Тема: «освоение возможностей языка программирования Pythone в разработке оконных приложений»

**Выполнил:**
Студент 3 курса
Группы ПО-12
Попов К. В.
Вариант 19

**Проверила:**
Кулик А. Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

**Задание 1. Построение графических примитивов и надписей**

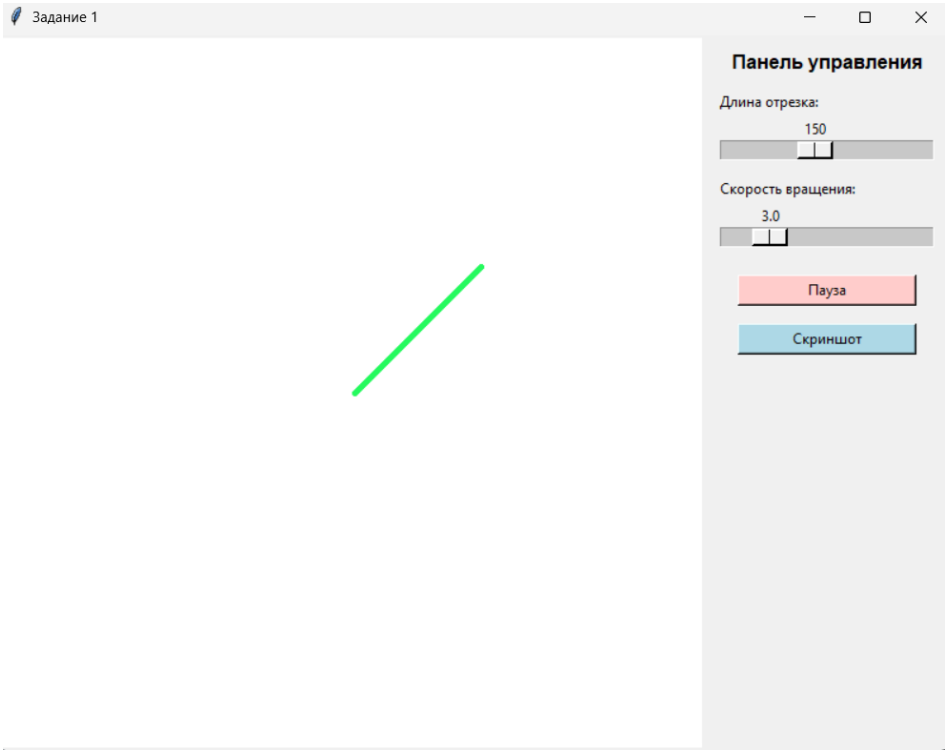
Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

5) Изобразить в окне приложения отрезок, вращающийся в плоскости экрана вокруг одной из своих концевых точек. Цвет прямой должен изменяться при переходе от одного положения к другому.




zadanie1.py

```python
"""
Модуль для лабораторной работы №7.
"""

import tkinter as tk
from tkinter import messagebox
import math
import time
import os
from PIL import ImageGrab


class RotatingLine:
    """Класс, описывающий вращающийся отрезок"""
    def __init__(self, canvas, x0, y0, length):
        self.canvas = canvas
        self.x0 = x0
        self.y0 = y0
        self.length = length
        self.angle = 0.0  # Текущий угол в градусах
        # Создаем линию на холсте
        self.line_id = self.canvas.create_line(x0, y0, x0, y0, width=5, capstyle=tk.ROUND)

    def set_length(self, length):
        """Изменение длины отрезка на лету"""
        self.length = length

    def step(self, speed):
        """Шаг вращения линии"""
        self.angle = (self.angle + speed) % 360
        self.draw()

    def get_color(self):
        """Вычисление цвета в зависимости от угла (плавный переход)"""
        r = int((math.sin(math.radians(self.angle)) + 1) * 127.5)
        g = int((math.sin(math.radians(self.angle + 120)) + 1) * 127.5)
        b = int((math.sin(math.radians(self.angle + 240)) + 1) * 127.5)
        return f"#{r:02x}{g:02x}{b:02x}"

    def draw(self):
        """Отрисовка отрезка на холсте в новом положении"""
```

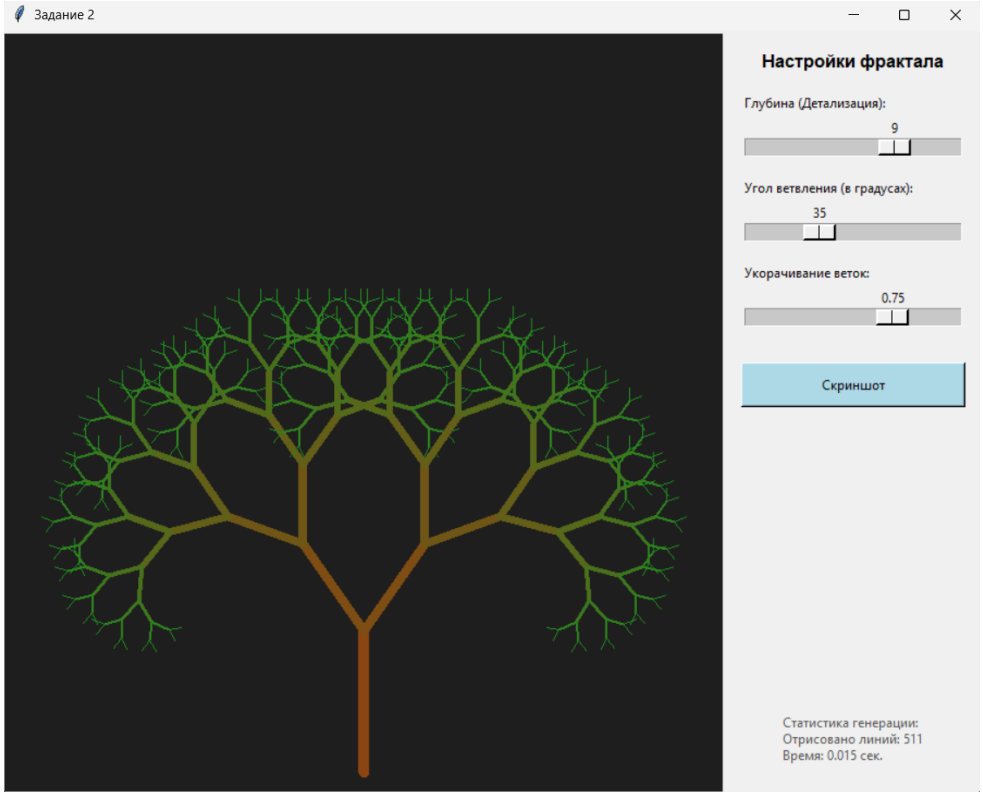
```
        rad = math.radians(self.angle)
        x1 = self.x0 + self.length * math.cos(rad)
        y1 = self.y0 + self.length * math.sin(rad)
        color = self.get_color()

        # Обновляем координаты и цвет
        self.canvas.coords(self.line_id, self.x0, self.y0, x1, y1)
        self.canvas.itemconfig(self.line_id, fill=color)



class App:
    """Главный класс приложения (интерфейс и управление)"""
    def __init__(self, root):
        self.root = root
        self.root.title("Задание 1")
        self.root.geometry("800x600")

        self.is_running = True
        self.length_var = tk.IntVar(value=150)
        self.speed_var = tk.DoubleVar(value=3.0)

        # Инициализируем переменные, чтобы избежать предупреждений Pylint
        self.canvas = None
        self.btn_pause = None
        self.line = None

        self.setup_ui()
        self.line = RotatingLine(self.canvas, 300, 300, self.length_var.get())
        self.animate()

    def setup_ui(self):
        """Настройка элементов интерфейса"""
        control_frame = tk.Frame(self.root, width=200, bg="#f0f0f0", padx=10, pady=10)
        control_frame.pack(side=tk.RIGHT, fill=tk.Y)

        self.canvas = tk.Canvas(self.root, bg="white")
        self.canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        tk.Label(
            control_frame, text="Панель управления",
            font=("Arial", 12, "bold"), bg="#f0f0f0"
```

```
    ).pack(pady=(0, 10))

    tk.Label(control_frame, text="Длина отрезка:", bg="#f0f0f0").pack(anchor="w")

    scale_length = tk.Scale(
        control_frame, from_=50, to=280, orient=tk.HORIZONTAL,
        variable=self.length_var, command=self.update_params,
        bg="#f0f0f0", length=180
    )
    scale_length.pack(pady=(0, 10))

    tk.Label(control_frame, text="Скорость вращения:", bg="#f0f0f0").pack(anchor="w")

    scale_speed = tk.Scale(
        control_frame, from_=0.5, to=15.0, resolution=0.5,
        orient=tk.HORIZONTAL, variable=self.speed_var,
        bg="#f0f0f0", length=180
    )
    scale_speed.pack(pady=(0, 10))

    self.btn_pause = tk.Button(
        control_frame, text="Пауза", command=self.toggle_pause,
        width=20, bg="#ffcccb"
    )
    self.btn_pause.pack(pady=10)

    # Сделана локальной переменной, чтобы не превышать лимит атрибутов класса
    btn_screenshot = tk.Button(
        control_frame, text="Скриншот", command=self.take_screenshot,
        width=20, bg="#add8e6"
    )
    btn_screenshot.pack(pady=5)

def update_params(self, *_args):
    """Обновление параметров 'на лету' при сдвиге ползунков"""
    if self.line:
        self.line.set_length(self.length_var.get())

def toggle_pause(self):
    """Переключение состояния анимации (Пауза/Работа)"""
    self.is_running = not self.is_running
```

```
        if self.is_running:
            self.btn_pause.config(text="Пауза", bg="#ffcccb")
            self.animate()
        else:
            self.btn_pause.config(text="Возобновить", bg="#ccffcc")

    def animate(self):
        """Цикл обновления кадров"""
        if self.is_running:
            self.line.step(self.speed_var.get())
            self.root.after(30, self.animate)

    def take_screenshot(self):
        """Создание скриншота активного окна"""
        x0 = self.root.winfo_rootx()
        y0 = self.root.winfo_rooty()
        x1 = x0 + self.root.winfo_width()
        y1 = y0 + self.root.winfo_height()

        try:
            img = ImageGrab.grab(bbox=(x0, y0, x1, y1))
            filename = f"screenshot_{int(time.time())}.png"
            filepath = os.path.join(os.getcwd(), filename)
            img.save(filepath)
            messagebox.showinfo("Успех", f"Скриншот сохранен:\n{filepath}")
        except OSError as error:
            messagebox.showerror("Ошибка", f"Не удалось сделать скриншот.\n{error}")


def main():
    """Главная функция для запуска приложения"""
    root = tk.Tk()
    _app = App(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

**Задание 2. Реализовать построение заданного типа фрактала по варианту**

5) Дерево Пифагора




Вывод: я освоил возможности языка программирования Python в разработке оконных приложений